國立臺北科技大學
National Taipei University of Technology

電子工程系碩士班

碩士學位論文

Department of Electronic Engineering

Master Thesis

非合作式 PCI 設備模擬

Non-cooperative PCI Device Emulation

研究生：蘇翊端

Researcher: Yi-Tuan Su

指導教授：賴冠廷 博士

Advisor: Kuan-Ting Lai, Ph.D.

中華民國一百一十五年七月

「學位論文口試委員會審定書」掃描檔

審定書填寫方式以系所規定為準，但檢附在電子論文內的掃描檔須具備以下條件：

1. 含指導教授、口試委員及系所主管的完整簽名。
2. 口試委員人數正確，碩士口試委員至少 3 人、博士口試委員至少 5 人。
3. 若此頁有論文題目，題目應和書背、封面、書名頁、摘要頁的題目相符。
4. 此頁有無浮水印皆可。

審定書不用頁碼

摘要

關鍵詞：虛擬化、設備模擬、VirtIO、PCI transport

輕量虛擬機器透過最小化映像檔來降低資源佔用與攻擊面，但同時移除了除錯與監控等工具。vmsh 針對此問題提出非合作式（non-cooperative）的解決方案，能在不修改虛擬機器監視器（hypervisor）與客體作業系統（Guest OS）的前提下，將 VirtIO 設備動態掛載至運行中的虛擬機器，為其補充所需的服務與工具。然而，vmsh 與 Cloud Hypervisor 在中斷機制上存在不相容，限制了 vmsh 的 hypervisor 支援範圍。

本研究擴充 vmsh 的功能，在維持非合作式與 hypervisor 無關（hypervisor-agnostic）的特性前提下，為 vmsh 實現了 VirtIO over PCI transport，並利用 KVM 所提供的 KVM_SIGNAL_MSI 介面實現 MSI-X 中斷的注入，來解決 vmsh 在中斷機制上與 Cloud Hypervisor 的不相容問題。在 PCI 設備模擬方面，本研究延伸 vmsh 的 side-load library 機制在客體作業系統內部觸發 PCI rescan，並透過 ptrace 攔截客體作業系統發出的 Port I/O 指令，使其得以發現由 vmsh 所模擬的 PCI 虛擬設備，進而透過標準 virtio-pci 驅動程式與該設備進行 I/O 操作。

此外，本研究在 QEMU 與 Cloud Hypervisor 兩種環境下完成功能驗證，並使用 xfstests 確認本研究中 PCI transport 實現的初步正確性，使 vmsh 成功支援 Cloud Hypervisor，擴展了 vmsh 的 hypervisor 支援範圍，並驗證了使用非合作式方法模擬 PCI 設備的可行性。

ABSTRACT

Keywords: Virtualization, Device Emulation, VirtIO, PCI Transport

Lightweight virtual machines reduce resource consumption and attack surface by minimizing their images, but at the cost of removing debugging and monitoring tools. vmsh addresses this problem with a non-cooperative approach that dynamically attaches VirtIO devices to running virtual machines without modifying the hypervisor or the guest OS, supplementing VMs with the required services and tools. However, vmsh is incompatible with Cloud Hypervisor due to differences in interrupt mechanisms, limiting the range of hypervisors that vmsh can support.

This study extends vmsh's functionality by implementing VirtIO over PCI transport while preserving its non-cooperative and hypervisor-agnostic properties. This study leverages the KVM_SIGNAL_MSI system call interface provided by KVM to inject MSI-X interrupts, resolving the interrupt mechanism incompatibility between vmsh and Cloud Hypervisor. For PCI device emulation, this study extends vmsh's side-load library mechanism to trigger PCI rescan within the guest OS, and intercepts Port I/O instructions issued by the guest via ptrace, enabling the guest OS to discover the PCI virtual device emulated by vmsh and perform I/O operations through the standard virtio-pci driver.

Furthermore, this study conducts functional verification under both QEMU and Cloud Hypervisor environments, and uses xfstests to confirm the preliminary correctness of the PCI transport implementation. As a result, vmsh successfully supports Cloud Hypervisor, expanding its hypervisor compatibility and demonstrating the feasibility of emulating PCI devices using a non-cooperative approach.

TAIPEI TECH

Acknowledgements

所有對於研究提供協助之人或機構，作者都可在誌謝中表達感謝之意。

TAIPEI TECH

Table of Contents

摘要 i

ABSTRACT ii

Acknowledgements iii

Table of Contents iv

List of Figures v

List of Tables vi

Chapter 1 Introduction 1

1.1 Background 1

1.2 Motivations 2

1.3 Research Contribution 3

1.4 Thesis Organization 4

Chapter 2 若標題太長，則可以分成兩行排列的形式撰寫 5

2.1 名詞定義（小標） 5

2.2 模型說明（小標） 5

2.3 VMSH: Hypervisor-agnostic Guest Overlays for VMs 6

2.3.1 System Architecture 6

Chapter 3 Experiment and Result 8

3.1 Functional Validation and Generality 8

3.1.1 PCI Device Discovery and Initialization 8

3.1.2 Block I/O and Filesystem Validation 8

3.1.3 Cloud Hypervisor Validation 8

References 11

TAIPEI TECH

List of Figures

2.1 Cool train station 5

2.2 System architecture of vmsh. vmsh establishes devices in the guest through side-loading a kernel library. The block device supports the overlay root file system, and the console device handles the I/O of the spawned process. Numbers 1–4 correspond to the four-step workflow. (Adapted from [1]) 6

3.1 dmesg log of PCI device discovery and BAR assignment (QEMU) 8

3.2 dmesg log of VirtIO negotiation completion (QEMU) 8

3.3 Guest userspace query of the vdb device 8

3.4 Block I/O read/write and data integrity test (QEMU) 9

3.5 ext4 filesystem mount test (QEMU) 9

3.6 Data integrity test (Cloud Hypervisor) 9

3.7 ext4 filesystem mount test (Cloud Hypervisor) 10

TAIPEI TECH

List of Tables

2.1 表格範例標題 5
2.2 Host machine specifications 7

TAIPEI TECH

Chapter 1 Introduction

1.1 Background

Virtualization is a cornerstone of modern cloud computing. Cloud service providers consolidate multiple tenants onto a single physical host through virtual machines (VMs) to achieve resource sharing and isolation. Among the various virtualization solutions, KVM (Kernel-based Virtual Machine) in the Linux kernel leverages hardware-assisted virtualization to provide efficient full virtualization capabilities and has become the dominant virtualization API in the industry, widely adopted by major cloud providers.

As the demand for performance-sensitive workloads grows, the industry has seen a trend toward designing lightweight VMs. These approaches reduce the VM image size to lower memory footprint and accelerate startup time, making them suitable for rapid-deployment scenarios such as serverless computing, while also improving system reliability and security by reducing the attack surface. The key to achieving this lightweight design lies in minimizing the VM's root filesystem, which entails removing services that are not needed during normal operation, such as monitoring and debugging tools.

However, this lightweight strategy introduces a fundamental tension: when users need to debug, monitor, or attach additional services to a VM, these tools no longer exist in the VM image. Rebuilding the image is not only cumbersome but also requires restarting the VM, causing the loss of runtime state and diagnostic information. This trade-off between the advantages of lightweight VMs and the availability of functionality raises a central question: is it possible to dynamically extend the capabilities of a running VM without modifying its image?

To address this problem, Thalheim et al. proposed vmsh [1] at EuroSys 2022, a hypervisor-agnostic mechanism for extending VM functionality. The core idea of vmsh is to dynamically attach VirtIO devices to a running VM from the outside, without modifying the hypervisor's source code or altering the guest image. Specifically, vmsh traces the hypervisor process using Linux's ptrace mechanism and leverages the KVM interface to side-load code into the guest kernel, registering VirtIO block and console devices within the guest to provide capabilities

such as filesystem mounting and interactive terminal access. At the device emulation level, vmsh employs VirtIO MMIO transport as the communication channel between host and guest, and uses KVM's irqfd mechanism for interrupt injection. This approach has been successfully validated on multiple mainstream KVM hypervisors, including QEMU, Firecracker, crosvm, and kvmtool, demonstrating its cross-hypervisor generality.

1.2 Motivations

Despite vmsh's demonstrated cross-hypervisor generality, a gap remains in its actual support coverage. Among the five KVM hypervisors it targets, Cloud Hypervisor, initiated by Intel, is the sole exception that remains incompatible. The authors of vmsh explicitly noted in their paper:

> *VMSH is able to support 4 out of the 5 hypervisors. Cloud Hypervisor is the exception as it uses PCIe's MSI-X messages for its interrupt handling. Therefore, it is incompatible with MMIO as a VirtIO transport channel.* [1]

vmsh's existing dynamic device attachment is implemented using VirtIO MMIO transport, whose interrupt injection relies on the interrupt routing mechanism internal to the KVM kernel. However, Cloud Hypervisor adopts a different interrupt architecture, rendering the interrupt path that vmsh depends on inoperable and consequently making the entire MMIO transport approach unusable on the Cloud Hypervisor platform.

To address this limitation, the authors of vmsh also outlined a research direction: "We plan to extend VMSH to support VirtIO over PCI for Cloud Hypervisor." [1] This implies that the virtual devices dynamically attached by vmsh would no longer be presented via MMIO transport but would instead appear as PCI devices within the guest OS, using MSI-X interrupt injection to replace the incompatible interrupt path and thereby resolving the compatibility issue with Cloud Hypervisor.

However, implementing VirtIO over PCI in a non-cooperative environment is fundamentally different from PCI device emulation performed internally by a hypervisor. In a typical

virtualization architecture, PCI device emulation is handled within the hypervisor, which has full access to PCI bus topology information and can directly manage device registration, configuration space responses, and interrupt routing. By contrast, vmsh operates outside the hypervisor and cannot access these internal data structures and interfaces; it must achieve the same objectives through indirect means. Specifically, as an external process independent of the hypervisor, vmsh faces the following technical challenges: first, intercepting the guest's accesses to PCI configuration space to enable virtual device discovery and configuration; second, injecting MSI-X interrupts to replace the incompatible interrupt path; and third, implementing complete VirtIO-PCI device emulation, including the Capabilities structure, BAR mapping, and virtqueue management.

In summary, the design of vmsh pursues two core objectives: non-cooperativeness, requiring no pre-deployed programs in the guest userspace, and generality, not relying on the internal interfaces of any specific hypervisor. As discussed above, vmsh's existing MMIO transport approach cannot support Cloud Hypervisor, while implementing PCI device emulation from outside the hypervisor involves several unresolved technical challenges. Therefore, how to implement VirtIO over PCI transport while maintaining vmsh's non-cooperative and hypervisor-agnostic design philosophy, thereby enabling vmsh to support Cloud Hypervisor, is the central research question that this study aims to address.

1.3 Research Contribution

This study extends vmsh to address its incompatibility with Cloud Hypervisor by implementing VirtIO over PCI transport in a non-cooperative environment. The main contributions are as follows:

Non-cooperative PCI device emulation. As an external process independent of the hypervisor, vmsh must complete device emulation from the outside without hypervisor cooperation. This study extends vmsh's side-load library mechanism to trigger a PCI rescan within the guest OS and intercepts Port I/O instructions issued by the guest via ptrace, implementing PCI device discovery, configuration space emulation, BAR mapping, and VirtIO-PCI negotiation, enabling

the guest to access the device emulated by vmsh through the standard virtio-pci driver.

Resolving interrupt mechanism compatibility and extending hypervisor support. vmsh's existing interrupt injection mechanism relies on the Full IRQChip architecture and cannot operate under Cloud Hypervisor's Split IRQChip environment. This study adopts the KVM_SIGNAL_MSI interface provided by KVM to implement MSI-X interrupt injection, making the interrupt mechanism independent of IRQChip architecture differences.

Cross-hypervisor validation of correctness and generality. Functional validation is performed under both QEMU and Cloud Hypervisor environments to confirm the generality of PCI transport; additionally, correctness testing using xfstests is conducted under the QEMU environment to confirm the preliminary correctness of PCI transport.

1.4 Thesis Organization

Chapter 1 Introduction presents the background and motivation of this study, describes the incompatibility between vmsh and Cloud Hypervisor, and states the contributions of this research.

Chapter ?? Literature Review reviews the technical background relevant to this study, including the KVM virtualization architecture, the VirtIO device model, the PCI bus and MSI-X interrupt mechanism, and introduces the system architecture and design of vmsh.

Chapter ?? Proposed Method details the system design and implementation of this study, covering PCI device discovery, Port I/O interception, VirtIO-PCI device emulation, and MSI-X interrupt injection.

Chapter ?? Experiment and Results presents the validation results of this study, including functional validation under QEMU and Cloud Hypervisor, correctness testing with xfstests, and a discussion of the limitations of the prototype implementation.

Chapter ?? Conclusion and Future Work summarizes the results and contributions of this study, and proposes future research directions including an asynchronous I/O model, performance testing, and integration with vmsh's broader functionality.

TAIPEI TECH

Chapter 2　若標題太長，則可以分成兩行排列的形式撰寫

2.1　名詞定義（小標）

定義定義定義定義定義定義 [2]，定義定義定義定義，定義定義定義定義定義定義定義定義定義定義，定義定義。

表 2.1: 表格範例標題

Protocol	P	CS_1	CS_2	RG
MSSMul	$O(1), O(1)$, N/A	$O(n-t), O(n), O(1)$	$O(n-t), O(n)$, N/A	$O(1), O(n), O(n)$
SC	$O(1), O(1)$, N/A	$O(n-t), O(n), O(1)$	$O(n-t), O(n)$, N/A	$O(1), O(n), O(n)$

2.2　模型說明（小標）

說明說明說明說明，說明說明說明說明說明說明說明說明說明說明說明說明說明，說明說明說明說明說明說明說明說明。

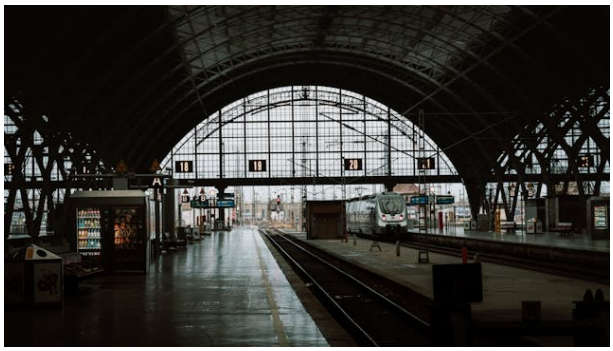

圖 2.1: Cool train station

2.3 VMSH: Hypervisor-agnostic Guest Overlays for VMs

2.3.1 System Architecture

As an independent host process, vmsh runs alongside the hypervisor on the same host machine. vmsh does not modify the hypervisor's source code, nor does it rely on any management API provided by the hypervisor. Instead, vmsh completes all operations externally through ptrace and the underlying KVM API. Figure 2.2 presents the system architecture of vmsh (adapted from [1] Figure 2), whose workflow can be divided into the following four steps:

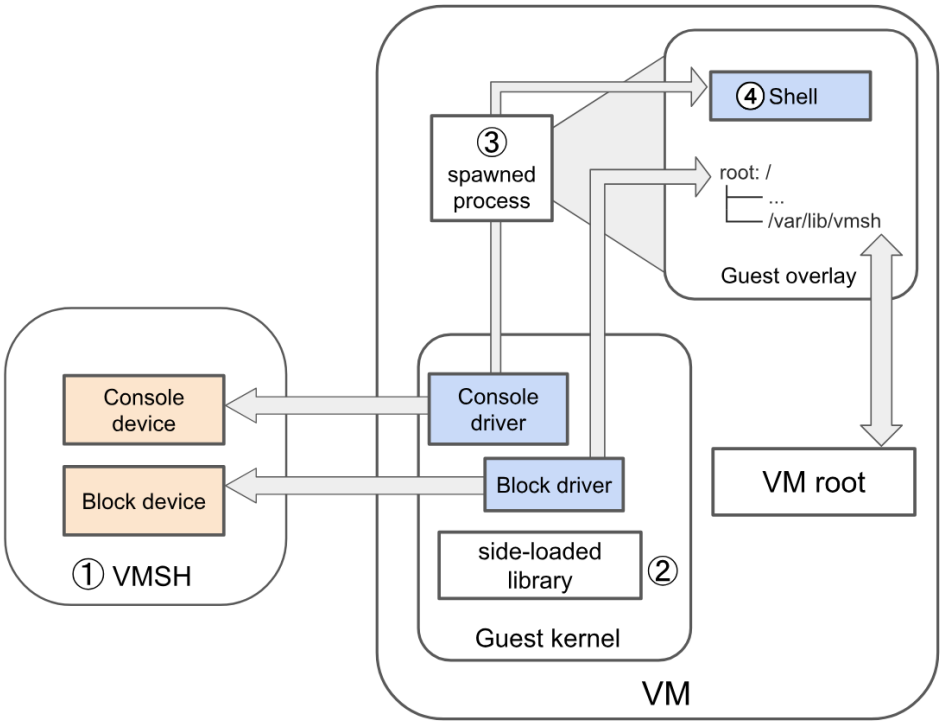

圖 2.2: System architecture of vmsh. vmsh establishes devices in the guest through side-loading a kernel library. The block device supports the overlay root file system, and the console device handles the I/O of the spawned process. Numbers 1–4 correspond to the four-step workflow. (Adapted from [1])

All experiments in this study were conducted on the same physical host machine, whose

hardware and operating system specifications are listed in Table 2.2.

表 2.2: Host machine specifications

Item	Specification
CPU	Intel Core i7-11700 @ 2.50GHz (8 cores, 16 threads)
Memory	48 GB DDR4
Storage	ADATA SX8200PNP 480GB NVMe SSD
OS	Ubuntu 22.04 LTS (kernel 5.15.0)

TAIPEI TECH

Chapter 3 Experiment and Result

3.1 Functional Validation and Generality

3.1.1 PCI Device Discovery and Initialization

```
[root@qemu-microvm:~]# dmesg | tail -25
[   47.007513] stage1: detected linux version 6.3.6
[   47.007514] stage1: using PCI transport (EXPERIMENTAL)
[   47.007514] stage1: [EXPERIMENTAL] triggering PCI bus rescan
[   47.009020] pci 0000:00:0a.0: [1af4:1042] type 00 class 0x010000
[   47.009710] pci 0000:00:0a.0: reg 0x10: [mem 0x00000000-0x00000fff]
[   47.010314] pci 0000:00:0a.0: reg 0x18: [mem 0x00000000-0x00000fff]
[   47.010921] pci 0000:00:0a.0: reg 0x20: [mem 0x00000000-0x00000fff]
[   47.011512] pci 0000:00:0a.0: reg 0x30: [mem 0x00000000-0x000007ff pref]
[   47.021659] resource: avoiding allocation from e820 entry [mem 0x0009fc00-0x000fffff]
[   47.021663] pci 0000:00:0a.0: BAR 0: assigned [mem 0x20000000-0x20000fff]
[   47.021777] resource: avoiding allocation from e820 entry [mem 0x0009fc00-0x000fffff]
[   47.021779] pci 0000:00:0a.0: BAR 2: assigned [mem 0x20001000-0x20001fff]
[   47.021897] resource: avoiding allocation from e820 entry [mem 0x0009fc00-0x000fffff]
[   47.021898] pci 0000:00:0a.0: BAR 4: assigned [mem 0x20002000-0x20002fff]
[   47.022007] resource: avoiding allocation from e820 entry [mem 0x0009fc00-0x000fffff]
[   47.022008] pci 0000:00:0a.0: BAR 6: assigned [mem 0x20003000-0x200037ff pref]
[   47.022869] virtio-pci 0000:00:0a.0: enabling device (0000 -> 0002)
```

圖 3.1: dmesg log of PCI device discovery and BAR assignment (QEMU)

```
[   47.052456] virtio_blk virtio2: [vdb] 131072 512-byte logical blocks (67.1 MB/64.0 MiB)
```

圖 3.2: dmesg log of VirtIO negotiation completion (QEMU)

```
[root@qemu-microvm:~]# ls /dev/vd*
/dev/vda  /dev/vdb

[root@qemu-microvm:~]# cat /sys/block/vdb/size
131072
```

圖 3.3: Guest userspace query of the vdb device

3.1.2 Block I/O and Filesystem Validation

3.1.3 Cloud Hypervisor Validation

```
[root@qemu-microvm:~]# dd if=/dev/vdb of=/dev/null bs=4k count=1
1+0 records in
1+0 records out
4096 bytes (4.1 kB, 4.0 KiB) copied, 0.00445993 s, 918 kB/s

[root@qemu-microvm:~]# dd if=/dev/zero of=/dev/vdb bs=4k count=1
1+0 records in
1+0 records out
4096 bytes (4.1 kB, 4.0 KiB) copied, 0.00578631 s, 708 kB/s

[root@qemu-microvm:~]# echo "VMSH_TEST_DATA" | dd of=/dev/vdb bs=512 count=1
0+1 records in
0+1 records out
15 bytes copied, 0.0124918 s, 1.2 kB/s

[root@qemu-microvm:~]# dd if=/dev/vdb bs=512 count=1 | head -c 20
1+0 records in
1+0 records out
VMSH_TEST_DATA
512 bytes copied, 0.00840111 s, 60.9 kB/s
```

圖 3.4: Block I/O read/write and data integrity test (QEMU)

```
[root@qemu-microvm:~]# lsblk
NAME
    MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
fd0   2:0    1     4K  0 disk
sr0  11:0    1  1024M  0 rom
vda 254:0    0 267.5M  1 disk /nix/store
                               /
vdb 254:16   0    10M  0 disk

[root@qemu-microvm:~]# mkdir -p /tmp/mnt

[root@qemu-microvm:~]# mount /dev/vdb /tmp/mnt

[root@qemu-microvm:~]# ls /tmp/mnt
bin  dev  etc  lost+found  nix  proc  sys  tmp  var

[root@qemu-microvm:~]# umount /tmp/mnt
```

圖 3.5: ext4 filesystem mount test (QEMU)

```
[root@qemu-microvm:~]# echo "VMSH_CH_TEST_2026" | dd of=/dev/vdb bs=512 count=1
0+1 records in
0+1 records out
18 bytes copied, 0.00511157 s, 3.5 kB/s

[root@qemu-microvm:~]# dd if=/dev/vdb bs=512 count=1 | head -c 20
1+0 records in
1+0 records out
VMSH_CH_TEST_2026
512 bytes copied, 0.00541082 s, 94.6 kB/s
```

圖 3.6: Data integrity test (Cloud Hypervisor)

```
[root@qemu-microvm:~]# lsblk
NAME
    MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
vda 254:0    0 267.5M  1 disk /nix/store
                              /
vdb 254:16   0    10M  0 disk

[root@qemu-microvm:~]# mkdir -p /tmp/mnt

[root@qemu-microvm:~]# mount /dev/vdb /tmp/mnt

[root@qemu-microvm:~]# ls /tmp/mnt
bin  dev  etc  lost+found  nix  proc  sys  tmp  var

[root@qemu-microvm:~]# umount /tmp/mnt

[root@qemu-microvm:~]# ls /tmp/mnt
```

圖 3.7: ext4 filesystem mount test (Cloud Hypervisor)

References

[1] Jörg Thalheim et al. “vmsh: Hypervisor-agnostic Guest Overlays for VMs”. In: *Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys ’22)*. Rennes, France: ACM, 2022, pp. 678–693.

[2] Leslie Lamport. *LaTeX: a Document Preparation System*. 2nd ed. Massachusetts: Addison Wesley, 1994.

TAIPEI TECH